База Python

Теория



Числа

- `int` без предела: `2 ** 100` считается точно. `float` — около 16 значащих цифр: `0.1 + 0.2` даёт `0.30000000000000004`. Вещественные сравнивают с допуском: `abs(a - b) < 1e-9` или `math.isclose(a, b)`.
- `/` всегда даёт `float`: `6 / 3 == 2.0`. Целое деление `//` и остаток `%` округляют вниз, к $-\infty$: `-7 // 2 == -4`, `-7 % 2 == 1`. Оба сразу — `divmod(a, b)`.
- `a ** b` — степень, `pow(a, b, m)` — степень по модулю, быстро даже при огромном b.
- `int("0042") == 42`. От вещественного `int` отбрасывает дробную часть, то есть округляет к нулю: `int(-3.9) == -3`.
- `bool` — это `int`: `True + True == 2`. Поэтому `sum(x > 0 for x in a)` — сколько положительных.
- `round` банковский: половина — к чётному, `round(2.5) == 2`, `round(0.125, 2) == 0.12`. А `round(2.675, 2) == 2.67`: в двоичной записи это 2,67499…, до половины не дотягивает. Школьное округление — через `Decimal` от строки:

```
from decimal import Decimal, ROUND_HALF_UP
Decimal(str(x)).quantize(Decimal("0.001"),
                         rounding=ROUND_HALF_UP)
```

- Ответ дробью — считать в `fractions.Fraction` с самого начала: `Fraction(6, 10)` печатается как `3/5`, сумма и произведение точные. `Fraction("0.25") == Fraction(1, 4)`.

Строки

- Индексы с нуля, отрицательные — с конца: `s[-1]` — последний символ. Срез `s[a:b]` — с a по $b-1$, `s[::-1]` — задом наперёд.
- Строка неизменяема: `s[0] = "x"` — ошибка. Каждый метод возвращает новую строку, старая не меняется.
- `s.split()` режет по пробелам, табуляциям и переводам строк, пустых кусков не даёт. `s.split(",")` режет ровно по запятой: `"a,,b".split(",") == ["a", "", "b"]`.
- `strip`, `lower`, `upper`, `replace(a, b)`, `startswith`, `count`. Не нашлось: `find` вернёт -1, `index` — ошибку.
- Склейка — `" ".join(parts)`, все части — строки: `" ".join(map(str, a))`. Много кусков — копить список и склеивать один раз.
- `isdigit` не пропускает минус и точку: `"-1".isdigit()` — ложь. Код символа — `ord("a") == 97`, обратно — `chr(97)`.
- Формат: `f"{x:.3f}"` — три знака после точки, `f"{n:05d}"` — `00042`, `f"{n:>5}"` — по правому краю в пять знаков, `f"{p:.1%}"` — в процентах.
- Строки сравниваются посимвольно: `"10" < "9"`, `"Z" < "a"`. Числа, прочитанные строками, сортируют с `key=int`.

Списки и кортежи

- `a[i]`, `append`, `pop()` — $O(1)$. `x in a`, `index`, `remove`, `pop(0)`, `insert(0, x)` — $O(n)$: проход по всему списку.
- `b = a` — тот же список под вторым именем, правка видна через оба. Копия — `a[:]` или `a.copy()`.
- Таблица $n \times m$ — `[[0] * m for _ in range(n)]`. Запись `[[0] * m] * n` даёт n ссылок на одну строку: правка одной клетки меняет весь столбец.
- `a.sort()` сортирует на месте и возвращает `None`; `sorted(a)` — новый список. `key=` — по чему сравнивать, `reverse=True` — по убыванию.
- Сортировка устойчива: равные по ключу остаются в прежнем порядке. Несколько ключей — кортежем, убывание числа — минусом:

  ```
  sorted(a, key=lambda p: (-p[1], p[0]))
  ```

  По второму полю по убыванию, при равенстве — по первому по возрастанию.
- Кортеж `(1, 2)` — неизменяемый список, годится ключом словаря. Распаковка: `a, b = b, a`, `first, *rest = a`.
- `enumerate(a)` — пары (номер, элемент), `zip(a, b)` — пары по позициям, по длине короткого.
- `min` и `max` тоже берут `key`. `max([])` — ошибка, `max(a, default=0)` — нет. Ещё под рукой: `sum`, `any`, `all`, `reversed`, `a.count(x)`.

Словари и множества

- `d[k]` для ключа, которого нет, — `KeyError`; `d.get(k, 0)` — со значением по умолчанию. `k in d` — $O(1)$.
- Обход — в порядке вставки: `for k, v in d.items()`. По значению:

```
sorted(d.items(), key=lambda kv: kv[1])
```

- `Counter(a)` — сколько раз встречается каждый элемент, отсутствующий — 0. `most_common(k)` — k самых частых, равные по счёту — в порядке первого появления.
- `defaultdict(list)`: `g[k].append(v)` без проверки, есть ли ключ. `defaultdict(int)` — счётчик.
- `set`: `x in s` за $O(1)$; `|`, `&`, `-`, `^` — объединение, пересечение, разность, симметрическая разность. Различных элементов — `len(set(a))`.
- Ключ словаря и элемент множества — неизменяемые: число, строка, кортеж. Список нельзя, кортеж из него — можно: `tuple(a)`.

Циклы и функции

- `range(a, b, k)` — от a до b, не включая b, с шагом k: `range(5, 0, -2)` — 5, 3, 1.
- `for ... else`: ветка `else` выполняется, если цикл не прервали `break`.
- Генераторы: список `[x * x for x in a if x > 0]`, словарь `{w: len(w) for w in ws}`, множество `{x % 3 for x in a}`. В `sum(x * x for x in a)` список не строится вовсе.
- `lambda x: ...` — функция в одно выражение, чаще всего для `key`.
- Значение по умолчанию вычисляется один раз, при объявлении: `def f(x, acc=[])` копит `acc` между вызовами. Правильно — `acc=None` и в начале тела `if acc is None: acc = []`.
- Имя, которому присваивают внутри функции, — локальное. Поменять внешнее — `nonlocal` (из объемлющей функции) или `global`. `a.append(x)` — не присваивание, работает и без них.
- Рекурсия глубже 1000 вызовов падает. Предел поднимает `sys.setrecursionlimit(10 ** 6)`, но на очень большой глубине питон может упасть и с ним — надёжнее цикл. Запомнить уже посчитанное — декоратор:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def f(n): ...
```

Ввод и вывод

Программа читает стандартный ввод и пишет в стандартный вывод. Число и строка чисел:

```
n = int(input())
a = list(map(int, input().split()))
```

n строк по два числа:

```
pts = [tuple(map(int, input().split()))
       for _ in range(n)]
```

Быстро и всё сразу:

```
import sys
data = sys.stdin.buffer.read().split()
n = int(data[0])
a = list(map(int, data[1:n + 1]))
```

`input()` в цикле на $2 \cdot 10^5$ строк ощутимо медленнее. Текст по строкам целиком — `sys.stdin.read().splitlines()`.

- `print(*a)` — через пробел, `print(*a, sep="\n")` — по строке, `end=""` — без перевода строки в конце.
- Много строк вывода — одной склейкой:

  ```
  sys.stdout.write("\n".join(map(str, ans))
                   + "\n")
  ```

- Вещественное — с нужным числом знаков: `print(f"{x:.6f}")`.
- В выводе ничего лишнего. Подсказка в `input("n = ")` тоже уходит в вывод. Отладку печатать в поток ошибок: `print(x, file=sys.stderr)`.

Скорость

- Питон делает порядка 10^7 простых операций в секунду. При $n = 2 \cdot 10^5$ двойной цикл — $4 \cdot 10^{10}$ шагов — не пройдёт, нужно $n \log n$ или n.
- $O(1)$: индекс в списке, `append`, `pop()`, `in` и запись в `set` и `dict`. $O(n)$: `in` по списку, `pop(0)`, `insert`, срез, `min`, `sum`. $O(n \log n)$: сортировка.
- `x in a` внутри цикла по большому списку — квадрат; сначала `s = set(a)`, потом `x in s`.
- Очередь — `deque`: `append` и `popleft` за $O(1)$. `list.pop(0)` в цикле — квадрат.
- Поиск в отсортированном — `bisect` за $O(\log n)$. Минимум меняющегося набора — куча `heapq`: добавить и достать за $O(\log n)$.
- Перед сдачей — крайние случаи: $n = 1$, все равны, отрицательные, самые большие числа.

Стандартная библиотека

- `math`: `gcd`, `lcm`, `comb(n, k)`, `factorial`, `isqrt` — целый корень без ошибок округления, `sqrt`, `log2`, `ceil`, `floor`, `inf`, `pi`.
- `itertools`: `product(a, repeat=k)` — все наборы длины k, `permutations`, `combinations(a, k)`, `accumulate` — префиксные суммы, `groupby` — группы *подряд* идущих равных, поэтому обычно после `sorted`.
- `collections`: `Counter`, `defaultdict`, `deque`.
- `bisect`: `bisect_left(a, x)` — первое место, куда можно вставить x без нарушения порядка, `bisect_right` — последнее. Их разность — сколько раз x встречается в отсортированном `a`.
- `heapq`: `heappush`, `heappop` — достаёт минимум. Куча максимумов — те же функции на $-x$. Ещё `nlargest(k, a)`.
- `functools`: `lru_cache`, `cmp_to_key` — сортировка по функции сравнения двух элементов.
- Версию питона на туре узнать заранее: `match` и `itertools.pairwise` появились только в 3.10.